

Matrix Chain Multiplication

Dynamic Programming III

(ADA-2022, CSE222)

Diptapriyo Majumdar

February 13, 2023

Matrix Chain Multiplication

- **Input:** A collection of matrices $A_1, \dots, A_n$ and we want to compute the product $A_1 A_2 A_3 \cdots A_n$. For every $i \in \{1, \dots, n\}$, A_i is of the order $a_{i-1} \times a_i$.
- **Goal:** Find an ordering (a way to parenthesize) for the matrix multiplications that minimizes the total number of scalar multiplications.
- **Example:** A, B, C are three matrices and to compute, there are two possible orderings, $(AB)C$ or $A(BC)$. Which particular ordering has the smallest number of scalar multiplications?

$$\begin{array}{l} \underline{\underline{(AB)C}} \\ 120 + 60 \\ = 180 \end{array}$$

$$\begin{array}{l} \underline{\underline{A(BC)}} \\ 72 + 90 \\ = 162 \end{array}$$

$$A_{5 \times 6} \quad B_{6 \times 4} \quad C_{4 \times 3}$$

$$5 \times 6 \quad 6 \times 3$$

Algorithm to Multiply two Matrices,

```
MULTIPLY( $A_{m \times n}, B_{n \times p}$ )  
for  $i = 1, \dots, m$  do  
    for  $j = 1, \dots, p$  do  
         $sum \leftarrow 0$ ;  
        for  $k = 1, \dots, n$  do  
             $sum \leftarrow sum + (A[i, k]B[k, j])$ ;  
        end  
         $C[i, j] \leftarrow sum$ ;  
    end  
end
```

Total number of multiplications needed is mnp .

Bracketing Examples

- Four matrices $A_{10 \times 6}$, $B_{6 \times 15}$, $C_{15 \times 8}$ and $D_{8 \times 4}$.
- $((AB)C)D$ requires $(10 \times 6 \times 15) + (10 \times 15 \times 8) + (10 \times 8 \times 4)$ scalar multiplications, i.e. $900 + 1200 + 320 = 2420$.
- $(A(BC))D$ requires $(6 \times 15 \times 8) + (10 \times 6 \times 8) + (10 \times 8 \times 4)$ scalar multiplications, i.e. $720 + 480 + 320 = 1520$.
- $((AB)(CD))$ requires $(10 \times 6 \times 15) + (15 \times 8 \times 4) + (10 \times 15 \times 4)$ scalar multiplications, i.e. $900 + 480 + 600 = 1980$.
- ✓ $(A(B(CD)))$ requires $(15 \times 8 \times 4) + (6 \times 15 \times 4) + (10 \times 6 \times 4)$ multiplications, i.e. $480 + 360 + 240 = 1080$.

$$(A(B(CD)))$$

How many ways can you parenthesize
 n matrices?

$A_1 A_2 \dots A_n \rightarrow \text{Computing}$

Counting bracketing

- How many ways can we do bracketing for n matrices, $A_1, A_2, \dots, A_n$ such that A_i is of order $p_{i-1} \times p_i$.

$$n=4$$

$$n=3$$

- Using the following recurrence.

- $P(n) = 1$ if $n = 0$, and

- $P(n) = \sum_{i=1}^{n-1} P(i)P(n-i-1)$ for all $n \geq 1$.

- This is related to the Catalan number.

- $P(n) \approx \frac{1}{n} \binom{2n-2}{n-1} \approx \frac{4^n}{n^{3/2}}$.

A brute force will need all possible bracketing.

- But we want an algorithm running polynomial in n .

Catalan Number.

$$P(1)P(n-2) + P(2)P(n-3) + \dots + P(n-1)P(0)$$

Understanding the structure

$$(A_i \cdot A_{i+1} \cdot A_{i+2} \cdots A_j) = A_{i,j}^*$$

- Let us use the notation $A_{i,j}^*$. The matrix $A_{i,j}^*$ is the matrix we get by multiplying $(A_i \cdot A_{i+1} \cdots A_j)$.
- Let $M[i, j]$ be the minimum number of scalar multiplications to get $A_{i,j}^*$. $\rightarrow$ subproblem
- $A_{i,j}^*$ is of order $p_{i-1} \times p_j$.
- If $j = i$, then no multiplication is needed. Hence, $M[i, i] = 0$.
- If $j > i$, then the product as to be broken as $(A_{i,k}^*) \cdot (A_{k+1,j}^*)$.

$$\begin{array}{ccccccc}
 A_i & & A_{i+1} & & \cdots & & A_j \\
 p_{i-1} \times p_i & & p_i \times p_{i+1} & & & & p_{j-1} \times p_j
 \end{array}$$

Base Case.

Splitting the bracketing

$$A_{i,j}^* = \underbrace{(A_i A_{i+1} A_{i+2} \dots A_k)}_{A_{i,k}^*} \underbrace{(A_{k+1} \dots A_j)}_{A_{k+1,j}^*}$$

- What is the total number of scalar multiplications for $(A_{i,k}^*)(A_{k+1,j}^*)$?
- The number of scalar multiplications to get $A_{i,k}^*$ plus the number of scalar multiplications to get $A_{k+1,j}^*$ plus the number of scalar multiplications to compute $A_{i,k}^* A_{k+1,j}^*$, i.e. $p_{i-1} p_k p_j$.

$$M[i,k] + M[k+1,j] + p_{i-1} p_k p_j$$

Optimal Substructure

- Suppose that to parenthesize $A_i \cdots A_j$, an optimal bracketing splits the product into $(A_i \cdots A_k) \cdot (A_{k+1} \cdots A_j)$. Then, the bracketing $B_{i,k}$ restricted to $(A_i \cdots A_k)$ is an optimal for $A_i \cdots A_k$ and the bracketing $B_{k+1,j}$ restricted to $(A_{k+1} \cdots A_j)$ is an optimal bracketing for $A_{k+1} \cdots A_j$.
- The above splitting requires $\text{mult}(B_{i,k}) + \text{mult}(B_{k+1,j}) + p_{i-1}p_kp_j$ multiplications.
- Why? Suppose not. *Proof by contradiction*
- Then, a different bracketing $C_{i,k}$ is an optimal bracketing restricted to $(A_i \cdots A_k)$ with less number of multiplications.
- Then, total number of multiplications is
 $\text{mult}(C_{i,k}) + \text{mult}(B_{k+1,j}) + p_{i-1}p_kp_j <$
 $\text{mult}(B_{i,k}) + \text{mult}(B_{k+1,j}) + p_{i-1}p_kp_j$. *+ mult(B_{k+1,j}) = OPT(C_{i,j})*

Parentthesize $(A_i A_{i+1} \dots A_j)$ -
An optimal bracketing splits this as
 $(A_i A_{i+1} \dots A_k) (A_{k+1} \dots A_j) \rightarrow (B_{i,k})$

Optimal bracketing restricted to $(A_i A_{i+1} \dots A_k)$
is $B_{i,k}$ and restricted to $(A_{k+1} \dots A_j)$
is $B_{k+1,j}$.

Suppose that $B_{i,k}$ is not an optimal
bracketing for $(A_i \dots A_k)$

Then a different bracketing $C_{i,k}$ is
an optimal bracketing for $(A_i \dots A_k)$.

Then $C_{i,k}$ combine with $B_{k+1,j}$ and
then multiply $A_{i,k}^* \cdot A_{k+1,j}^*$
is a valid bracketing for $A_{i,j}^*$

This different bracketing needs

scalar multiplications for $C_{i,k}$
+
scalar multiplications for $B_{k+1,j}$

$$\begin{matrix} & & + \\ p_i & p_k & p_j \end{matrix} \quad \text{scalar multiplications}$$

This new bracketing requires
less multiplications than

$$B_{i,k} \text{ combine with } B_{k+1,j}$$

This contradicts. that original
bracketing is optimal.

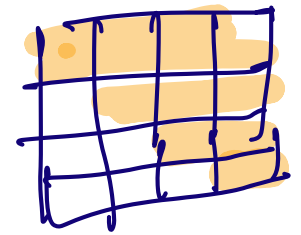
Recurrence

- If we split $A_i \cdots A_j$ by $(A_i \cdots A_k) \cdot (A_{k+1} \cdots A_j)$ in an optimal solution, then

$$M[i, j] = M[i, k] + M[k + 1, j] + p_{i-1}p_kp_j.$$

- The recurrence is

$$M(i, j) = \begin{cases} 0 & \text{if } i = j \\ \min_{i \leq k < j} \{M[i, k] + M[k + 1, j] + p_{i-1}p_kp_j\} & \text{otherwise} \end{cases}$$



- The table can be filled up incrementally.

Original problem is $A_1 \cdots A_n$

Solution to the original problem is

$$M[1, n].$$

Filling up the table

$A_1, \dots, A_n$. Order of A_i is $p_{i-1} \times p_i$.

- For all $i \in \{1, \dots, n\}$, $M[i, i] = 0$.
- Keep filling up the entries $M[1, 2], M[2, 3], \dots, M[n-1, n]$ in this order.
- $M[i, i+1] = M[i, i] + M[i+1, i+1] + p_{i-1}p_i p_{i+1}$.
- Then fill up $M[1, 3], M[2, 4], \dots, M[n-2, n]$ and so on.
- **Example:** Four matrices $A_{10 \times 6}, B_{6 \times 15}, C_{15 \times 8}$ and $D_{8 \times 4}$.

Next slide

$$p_0 = 10, p_1 = 6, p_2 = 15, p_3 = 8, p_4 = 4$$

Four matrices $A_{10 \times 6}$, $B_{6 \times 15}$, $C_{15 \times 8}$ and $D_{8 \times 4}$. The recurrence is

$$M(i, j) = \begin{cases} 0 & \text{if } i = j \\ \min_{i \leq k < j} \{M[i, k] + M[k+1, j] + p_{i-1}p_kp_j\} & \text{otherwise} \end{cases}$$

$$\begin{aligned} M[1, 3] &= ? \\ M[1, 1] + M[2, 3] &+ 480 \\ &= 1200 \\ M[1, 2] + M[3, 3] &+ p_0 p_2 p_3 \\ &= 900 + 0 + 1200 \end{aligned}$$

	1	2	3	4
A_1	0	900	1200	1080
A_2		0	720	810
A_3			0	480
A_4				0

$$\begin{aligned} M[2, 4] &= p_1 p_2 p_4 \\ &+ M[2, 2] \\ &+ M[3, 4] \\ &= 840 \end{aligned}$$

$$\begin{aligned} M[2, 3] + M[4, 4] &+ p_1 p_3 p_4 = 912 \end{aligned}$$

$$M[1,1] + M[2,4] + p_0 p_1 p_4 = 1080$$

$$M[1,2] + M[3,4] + p_0 p_2 p_4 = 1380 +$$

$$M[1,3] + M[4,4] + p_0 p_3 p_4 =$$

$$= 1200 + 480 +$$

Algorithm

Matrix-Chain-Order($A_1, \dots, A_n$)

for $i = 1, \dots, n$ do

 | $M[i, i] = 0;$

end

for $\ell = 2, \dots, n$ do

 for $i = 1, \dots, n - \ell + 1$ do

$j = i + \ell - 1; M[i, j] = \infty;$

 for $k = i, \dots, j - 1$ do

$q = M[i, k] + M[k + 1, j] + p_{i-1}p_kp_j;$

 if $q < M[i, j]$ then

 | $M[i, j] = q;$

 end

 end

 end

end

Return $M[1, n]$

$(n-1) (n-2) (n-3)$

$O(n^3)$

- Introduction to Algorithms by CLRS – Chapter 14.2.